

计算机视觉作业 3 实验报告：图像拼接算法实现

1. 实验概述

本实验实现了一个完整的图像拼接系统，包含两个主要任务：

1. **两图像拼接**：实现基础的图像拼接流程，包括特征对应、单应性计算、图像变换和马赛克创建
2. **多图像拼接**：设计鲁棒的多图像拼接流水线，能够处理任意数量的输入图像

2. 算法实现

2.1 任务 1：两图像拼接算法

2.1.1 系统架构

```
class TwoImageStitcher:
    def __init__(self):
        self.sift = cv2.SIFT_create(nfeatures=1000)
        self.orb = cv2.ORB_create(nfeatures=1000)
```

2.1.2 核心算法组件

1. 特征对应点获取

```
def get_correspondences(self, img1, img2):
```

- 优先使用 SIFT 检测器（1000个特征点）进行特征检测，备用 ORB 检测器
- 使用对应的描述子算法（SIFT / ORB）计算描述子
 - SIFT：使用 FLANN（快速最近邻）匹配器
 - ORB：使用 BFMatcher（暴力匹配器）with Hamming 距离
- 用 Lowe's ratio test（阈值0.7）进行匹配筛选，过滤不可靠匹配。对于特征点 f_i ，设其最近邻距离为 d_1 ，次近邻距离为 d_2 ，则：

$$\frac{d_1}{d_2} < \tau$$

其中 $\tau = 0.7$ 为阈值

2. 单应性矩阵计算

```
def compute_homography(self, src_pts, dst_pts):
```

- 使用 OpenCV 的 `cv2.findHomography()`
- RANSAC 算法，重投影阈值 5.0 像素
- 计算内点数量和内点比例
- 求解 3×3 单应性矩阵 H ，满足 $\lambda p'_i = H p_i$

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

其中 (x', y') 和 (x, y) 分别为两幅图像中的对应点坐标

3. 图像变换

```
def warp_image(self, img, H, reference_shape):
```

- 使用 `cv2.warpPerspective()` 进行逆向变换
- 计算变换后图像的包围盒
- 自动确定能容纳两幅图像的最小画布
- 处理负坐标，确保所有像素都在有效范围内

4. 马赛克创建

```
def create_mosaic(self, img1, img2, warped_img1, offset, output_size):
```

- 首先放置变换后的图像，然后覆盖参考图像
- 只在有效像素的区域进行覆盖
- 直接覆盖策略，黑色区域表示无数据

2.2 任务 2：多图像拼接算法

2.2.1 系统架构设计

```
class MultiImageStitcher:
    def __init__(self):
        self.sift = cv2.SIFT_create(nfeatures=1500, contrastThreshold=0.04, edgeThreshold=12)
        self.orb = cv2.ORB_create(nfeatures=1000)
        self.ratio_threshold = 0.8
        self.min_match_count = 10
        self.min_inlier_ratio = 0.15
```

2.2.2 核心技术组件

1. 图像预处理

```
def preprocess_image(self, img):
```

- 使用CLAHE（限制对比度自适应直方图均衡化），增强对比度
- 参数设置为 `clipLimit=1.5`，`tileGridSize=(8,8)`

2. 鲁棒特征匹配

```
def robust_feature_matching(self, img1, img2):
```

- 使用多特征策略，SIFT（1500特征）+ ORB（1000特征）备用
- 根据特征类型选择合适的匹配器
- 质量控制：Lowe's ratio test + 最小匹配数量阈值

3. 改进的单应性估计

```
def robust_homography_estimation(self, src_pts, dst_pts):
```

- 使用RANSAC，尝试不同的阈值和迭代次数组合
 - 阈值范围：3.0-5.0像素
 - 迭代次数：1000-2000次
 - 置信度：0.9-0.99
- 选择内点比例最高的结果
- **RANSAC 算法**：对于给定的匹配点集，RANSAC 通过以下步骤估计单应性：
 1. 随机选择4个点对计算单应性矩阵
 2. 计算所有点的重投影误差
 3. 统计误差小于阈值 τ 的内点数量
 4. 重复步骤1-3，选择内点数量最多的结果

4. 单应性验证

```
def validate_homography(self, H, img_shape, inlier_ratio):
```

- 检查变换后的图像形状合理性
- 限制面积变化在 0.05-20 倍之间
- 限制最大边长与最小边长比例 < 10
- 要求最低 10% 的内点比例

5. 图像匹配图构建

```
def build_matching_graph(self, images):
```

- 使用 NetworkX 构建无向图 $G = (V, E)$
- 每个图像 I_i 作为一个节点 $v_i \in V$
- 使用内点数量作为连接强度 $w_{ij} = |inliers_{ij}|$
- 确保图的连通性以支持全局对齐

6. 束调整优化

```
def bundle_adjustment_conservative(self, images, G, match_results, reference):
```

- **初始化**：使用 BFS 从参考图像计算初始单应性
- **目标函数**：束调整最小化所有图像对的重投影误差：

$$E = \sum_{(i,j) \in E} \sum_{k=1}^{n_{ij}} \|p_{jk} - \pi(H_j^{-1} H_i p_{ik})\|^2$$

其中 p_{ik} 和 p_{jk} 是图像 i 和 j 中的对应点， H_i 是图像 i 到参考坐标系的单应性矩阵

- **优化器**：SciPy 的 `least_squares` 求解器
- **保守策略**：限制迭代次数避免过拟合
- **参数化**：8 参数单应性矩阵（固定第 9 个参数为 1）

7. 改进的图像融合

```
def improved_blending(self, img1, mask1, img2, mask2):
```

- **重叠检测**：自动识别图像重叠区域
- **距离变换**：计算到边界的距离用于权重计算

- **加权融合**：在重叠区域使用距离加权平均。对于重叠像素 (x, y) ，其融合权重为：

$$w_1(x, y) = \frac{d_1(x, y)}{d_1(x, y) + d_2(x, y)}, \quad w_2(x, y) = \frac{d_2(x, y)}{d_1(x, y) + d_2(x, y)}$$

其中 $d_i(x, y)$ 是像素到图像 i 边界的距离

- **边界处理**：非重叠区域直接复制

2.2.3 拼接流程设计

```
def create_mosaic(self, images):
```

1. **图像缩放**：限制最大尺寸为 1000 像素，提高处理效率
2. **匹配图构建**：计算所有图像对的匹配关系
3. **连通性检查**：处理不连通图，选择最大连通分量
4. **参考图像选择**：选择连接度最高的图像作为参考
5. **束调整优化**：全局优化所有单应性矩阵
6. **画布计算**：确定能容纳所有图像的最小画布
7. **顺序拼接**：按连接强度顺序逐个拼接图像

3. 算法分析

1. 优势

- 算法的鲁棒性强
- 束调整优化显著提升了几何一致性
- 能处理不同类型和数量的图像
- 在质量和速度间找到了良好的平衡

2. 局限性

- 对于大视差、遮挡严重的场景仍有存在问题
- 大尺寸图像处理需要较多内存